Seatwork 10.1

Group Members: Ruzyl Bryan V. Amora, Alvin John Dano Section: CPE22S3

Extract:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

df = pd.read_csv('/content/drive/MyDrive/RT_IOT2022.csv')

print(df.shape)
df.head()
```

```
(123117, 85)
```

```
{"type":"dataframe","variable_name":"df"}
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

Transform:

```
print("Missing values per column:\n", df.isnull().sum())

df = df.dropna(subset=['Attack_type'])

df = df.drop_duplicates()

print("\nAfter cleaning, shape:", df.shape)

print("\nData types:\n", df.dtypes)
```

```
Missing values per column:
 no                        0
id.orig_p                  0
id.resp_p                  0
proto                      0
service                    0
                          ..
idle.std                   0
fwd_init_window_size       0
bwd_init_window_size       0
fwd_last_window_size       0
```

```
Attack_type              0
Length: 85, dtype: int64

After cleaning, shape: (123117, 85)

Data types:
 no                        int64
id.orig_p                  int64
id.resp_p                  int64
proto                     object
service                   object
                           ...
idle.std                  float64
fwd_init_window_size        int64
bwd_init_window_size        int64
fwd_last_window_size        int64
Attack_type               object
Length: 85, dtype: object
```

Load:

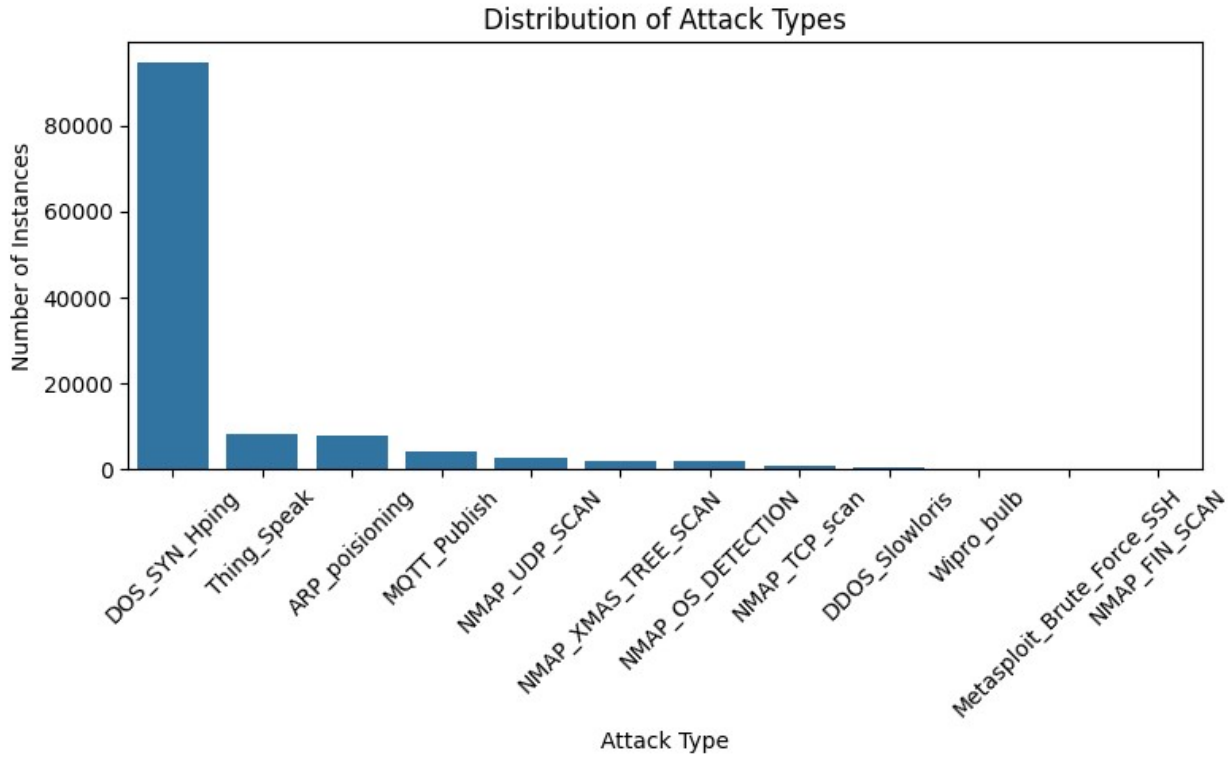
```
df.to_csv('RT_IoT2022_cleaned.csv', index=False)
```

Guide Questions:

What is the distribution of the Attack_type classes (normal vs. various attacks), and what percentage of the 123,117 instances does each class comprise?

```
attack_counts = df['Attack_type'].value_counts()
attack_percent = df['Attack_type'].value_counts(normalize=True) * 100

summary = pd.DataFrame({
    'Count': attack_counts,
    'Percentage': attack_percent.round(2)
})
print("Attack type distribution:")
print(summary)

plt.figure(figsize=(8,5))
sns.barplot(x=attack_counts.index, y=attack_counts.values)
plt.title('Distribution of Attack Types')
plt.xlabel('Attack Type')
plt.ylabel('Number of Instances')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

Attack type distribution:
                            Count  Percentage
```

```
Attack_type
DOS_SYN_Hping                94659        76.89
Thing_Speak                   8108         6.59
ARP_poisioning                7750         6.29
MQTT_Publish                  4146         3.37
NMAP_UDP_SCAN                 2590         2.10
NMAP_XMAS_TREE_SCAN           2010         1.63
NMAP_OS_DETECTION             2000         1.62
NMAP_TCP_scan                 1002         0.81
DDOS_Slowloris                 534         0.43
Wipro_bulb                     253         0.21
Metasploit_Brute_Force_SSH      37         0.03
NMAP_FIN_SCAN                   28         0.02
```



How do the categorical features proto (protocol) and service vary across different attack types and normal traffic patterns? Links to an external site.

```
print("Unique protocols:", df['proto'].unique())
print("Unique services:", df['service'].unique())

proto_attack = pd.crosstab(df['proto'], df['Attack_type'],
normalize='index') * 100
print("\nProtocol distribution (%) per Attack Type:")
print(proto_attack.round(2))

service_attack = pd.crosstab(df['service'], df['Attack_type'],
```

```
normalize='index') * 100
print("\nService distribution (%) per Attack Type:")
print(service_attack.round(2))

Unique protocols: ['tcp' 'udp' 'icmp']
Unique services: ['mqtt' '-' 'http' 'dns' 'ntp' 'ssl' 'dhcp' 'irc'
'ssh' 'radius']

Protocol distribution (%) per Attack Type:
Attack_type  ARP_poisioning  DDOS_Slowloris  DOS_SYN_Hping
MQTT_Publish  \
proto

icmp                   14.04            0.00           0.00
0.00
tcp                     1.75            0.48          85.72
3.75
udp                    46.03            0.04           0.00
0.00

Attack_type  Metasploit_Brute_Force_SSH  NMAP_FIN_SCAN
NMAP_OS_DETECTION  \
proto

icmp                                 0.00           0.00
0.00
tcp                                  0.03           0.02
1.81
udp                                  0.06           0.02
0.00

Attack_type  NMAP_TCP_scan  NMAP_UDP_SCAN  NMAP_XMAS_TREE_SCAN
Thing_Speak  \
proto

icmp                   0.00           3.51                 0.00
78.95
tcp                    0.91           0.13                 1.82
3.42
udp                    0.00          19.36                 0.02
33.91

Attack_type  Wipro_bulb
proto
icmp               3.51
tcp                0.16
udp                0.55

Service distribution (%) per Attack Type:
Attack_type  ARP_poisioning  DDOS_Slowloris  DOS_SYN_Hping
```

```
MQTT_Publish  \
service

-                        0.53             0.01           92.03
0.01
dhcp                    52.00             4.00            0.00
0.00
dns                     57.24             0.03            0.00
0.00
http                     3.72            15.10            0.00
0.09
irc                      0.00             0.00            0.00
0.00
mqtt                     0.00             0.00            0.00
100.00
ntp                      5.79             0.00            0.00
0.00
radius                   0.00             0.00            0.00
0.00
ssh                      0.00             0.00            0.00
0.00
ssl                     54.79             0.00            0.00
0.00

Attack_type  Metasploit_Brute_Force_SSH  NMAP_FIN_SCAN
NMAP_OS_DETECTION  \
service

-                                  0.00            0.02
1.94
dhcp                               0.00            0.00
0.00
dns                                0.08            0.03
0.00
http                               0.03            0.03
0.00
irc                                0.00            0.00
0.00
mqtt                               0.00            0.00
0.00
ntp                                0.00            0.00
0.00
radius                             0.00            0.00
0.00
ssh                              100.00            0.00
0.00
ssl                                0.00            0.00
0.00

Attack_type  NMAP_TCP_scan  NMAP_UDP_SCAN  NMAP_XMAS_TREE_SCAN
```

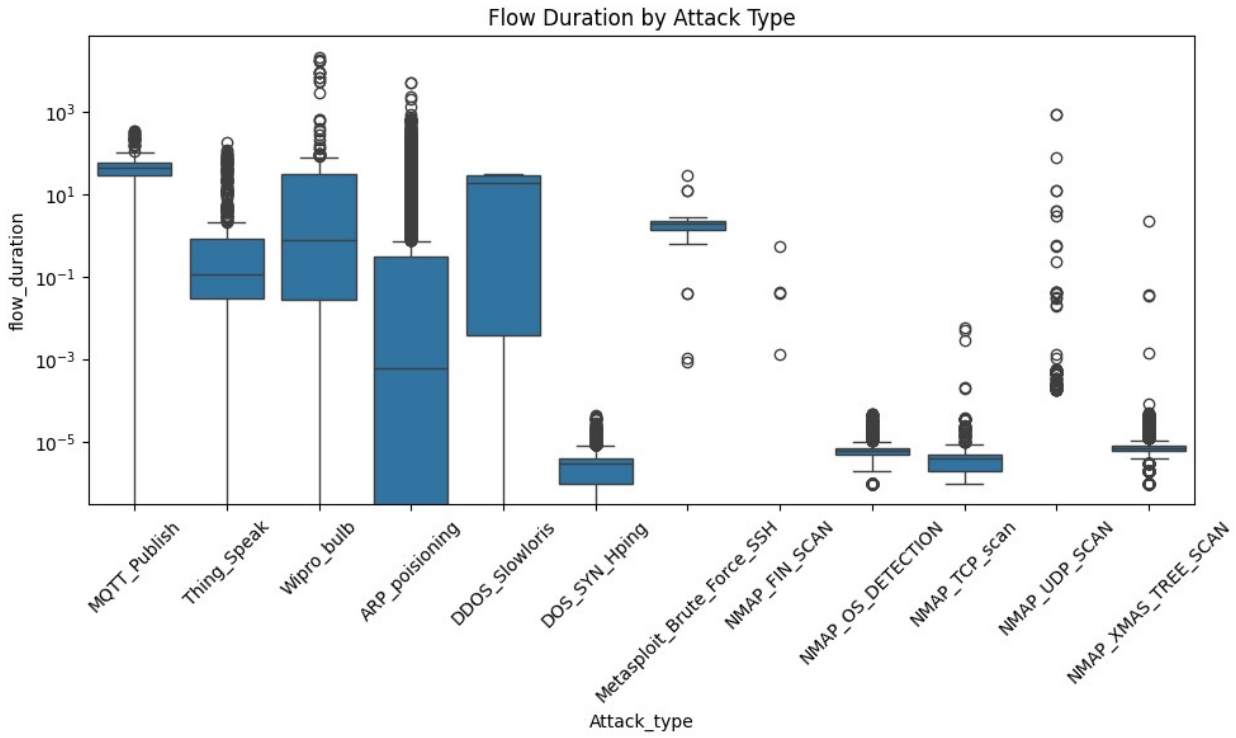
```
Thing_Speak  \
service

-                      0.97           2.34                   1.95
0.15
dhcp                   0.00           6.00                   0.00
28.00
dns                    0.00           0.32                   0.03
41.72
http                   0.00           4.01                   0.03
76.99
irc                    0.00           0.00                   0.00
0.00
mqtt                   0.00           0.00                   0.00
0.00
ntp                    0.00           3.31                   0.00
90.91
radius                 0.00         100.00                   0.00
0.00
ssh                    0.00           0.00                   0.00
0.00
ssl                    0.00           0.00                   0.00
41.19

Attack_type  Wipro_bulb
service
-                   0.04
dhcp               10.00
dns                 0.54
http                0.00
irc               100.00
mqtt                0.00
ntp                 0.00
radius              0.00
ssh                 0.00
ssl                 4.02
```

What are the mean and standard deviation of flow_duration for each Attack_type, and are differences statistically significant?

```
duration_stats = df.groupby('Attack_type')
['flow_duration'].agg(['mean', 'std']).round(2)
print("Mean and standard deviation of flow_duration per attack type:")
print(duration_stats)

plt.figure(figsize=(10,6))
sns.boxplot(x='Attack_type', y='flow_duration', data=df)
plt.title('Flow Duration by Attack Type')
plt.xticks(rotation=45)
```

```
plt.yscale('log')
plt.tight_layout()
plt.show()

df_duration = df.dropna(subset=['flow_duration'])
groups = [group['flow_duration'].values for name, group in
df_duration.groupby('Attack_type')]
f_stat, p_value = stats.f_oneway(*groups)

print(f"\nANOVA result: F-statistic = {f_stat:.3f}, p-value =
{p_value:.3e}")
if p_value < 0.05:
    print("The differences in flow duration among attack types are
statistically significant.")
else:
    print("No significant difference in flow duration among attack
types.")
```

```
Mean and standard deviation of flow_duration per attack type:
                              mean      std
Attack_type
ARP_poisioning               15.89   108.26
DDOS_Slowloris               14.70    14.12
DOS_SYN_Hping                 0.00     0.00
MQTT_Publish                 43.40    24.34
Metasploit_Brute_Force_SSH    3.01     5.21
NMAP_FIN_SCAN                 0.02     0.11
NMAP_OS_DETECTION             0.00     0.00
NMAP_TCP_scan                 0.00     0.00
NMAP_UDP_SCAN                 0.74    24.91
NMAP_XMAS_TREE_SCAN           0.00     0.05
Thing_Speak                   0.93     5.25
Wipro_bulb                  586.85  2738.89
```




```
ANOVA result: F-statistic = 536.761, p-value = 0.000e+00
The differences in flow duration among attack types are statistically
significant.
```

Which continuous features (e.g., fwd_pkts_per_sec, bwd_pkts_per_sec, payload_bytes_per_second) exhibit the highest correlation with specific attack classes?

```python
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['Attack_type_encoded'] = le.fit_transform(df['Attack_type'])

continuous_features = [
    'fwd_pkts_per_sec',
    'bwd_pkts_per_sec',
    'payload_bytes_per_second',
    'flow_duration',
    'flow_pkts_per_sec'
]

correlation = df[continuous_features + ['Attack_type_encoded']].corr()

corr_with_target =
correlation['Attack_type_encoded'].sort_values(ascending=False)
print(corr_with_target)
```

```
Attack_type_encoded          1.000000
flow_duration                0.024384
fwd_pkts_per_sec            -0.251517
flow_pkts_per_sec           -0.251550
bwd_pkts_per_sec            -0.251580
payload_bytes_per_second    -0.296198
Name: Attack_type_encoded, dtype: float64
```

How do time-based features like fwd_iat.avg and bwd_iat.avg (mean inter-arrival times) differ between various attack types and normal traffic?

```
df['Attack_type'] = df['Attack_type'].astype('category')
time_features = ['fwd_iat.avg', 'bwd_iat.avg']

# 1. DESCRIPTIVE STATISTICS
time_stats = df.groupby('Attack_type')[time_features].agg(['mean',
'std'])
print(time_stats)

# 2. BOXPLOT
plt.figure(figsize=(12,5))

# Forward IAT
plt.subplot(1,2,1)
sns.boxplot(x='Attack_type', y='fwd_iat.avg', data=df)
plt.xticks(rotation=45)
plt.title('Forward IAT Average by Attack Type')

# Backward IAT
plt.subplot(1,2,2)
sns.boxplot(x='Attack_type', y='bwd_iat.avg', data=df)
plt.xticks(rotation=45)
plt.title('Backward IAT Average by Attack Type')

plt.tight_layout()
plt.show()

# 3. BAR CHART (MEAN VALUES)
mean_values = df.groupby('Attack_type')[time_features].mean()

mean_values.plot(kind='bar', figsize=(12,6))
plt.title('Mean IAT Values per Attack Type')
plt.ylabel('Average IAT')
plt.xticks(rotation=45)
plt.legend(title="Features")
plt.tight_layout()
plt.show()

plt.figure(figsize=(10,5))
sns.kdeplot(data=df, x='fwd_iat.avg', hue='Attack_type', fill=True)
```

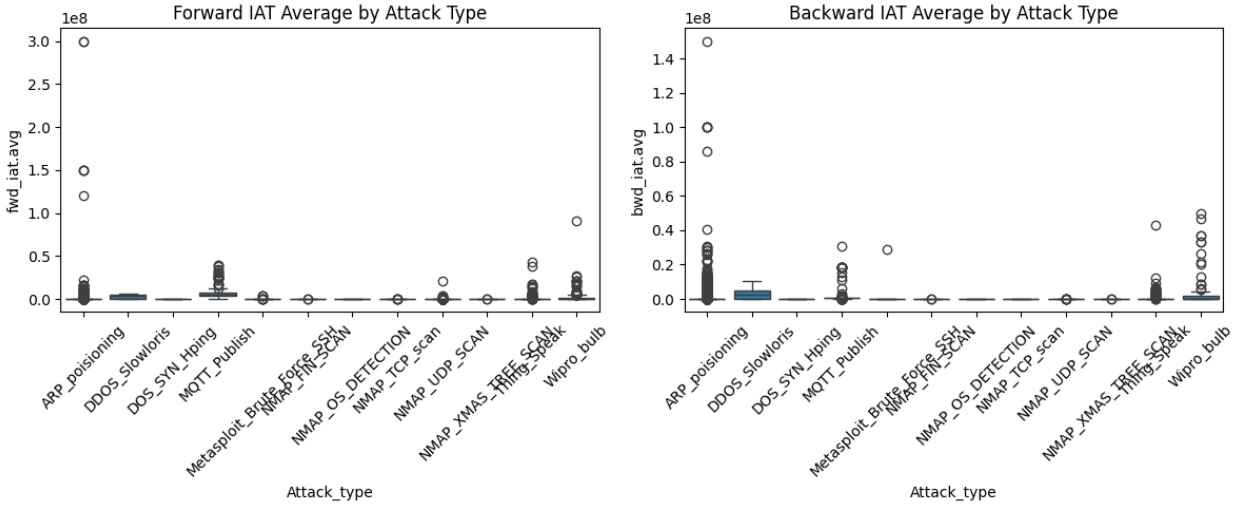
```
plt.title('Distribution of Forward IAT Average')
plt.show()

plt.figure(figsize=(10,5))
sns.kdeplot(data=df, x='bwd_iat.avg', hue='Attack_type', fill=True)
plt.title('Distribution of Backward IAT Average')
plt.show()
```

```
/tmp/ipykernel_12991/3025982171.py:5: FutureWarning: The default of
observed=False is deprecated and will be changed to True in a future
version of pandas. Pass observed=False to retain current behavior or
observed=True to adopt the future default and silence this warning.
  time_stats = df.groupby('Attack_type')[time_features].agg(['mean',
'std'])
```

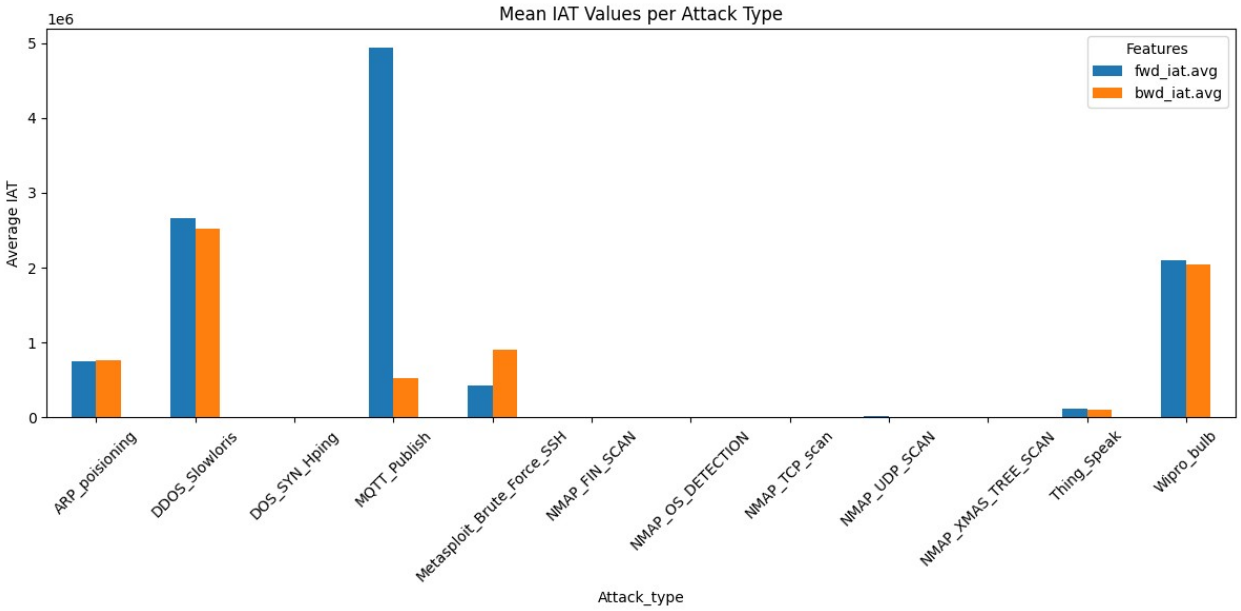
	fwd_iat.avg		bwd_iat.avg
	mean	std	mean
Attack_type			
ARP_poisioning	7.443455e+05	6.080604e+06	7.664256e+05
DDOS_Slowloris	2.660823e+06	2.505477e+06	2.523958e+06
DOS_SYN_Hping	0.000000e+00	0.000000e+00	0.000000e+00
MQTT_Publish	4.941675e+06	2.703778e+06	5.222072e+05
Metasploit_Brute_Force_SSH	4.313227e+05	1.071899e+06	9.100383e+05
NMAP_FIN_SCAN	4.114728e+03	2.176702e+04	6.539123e+03
NMAP_OS_DETECTION	0.000000e+00	0.000000e+00	0.000000e+00
NMAP_TCP_scan	7.311979e+00	1.345700e+02	0.000000e+00
NMAP_UDP_SCAN	1.436467e+04	4.294105e+05	1.786866e+02
NMAP_XMAS_TREE_SCAN	1.874295e+02	8.402341e+03	1.716461e+02
Thing_Speak	1.186232e+05	8.328482e+05	9.726068e+04
Wipro_bulb	2.093047e+06	7.004656e+06	2.037756e+06

	std
Attack_type	
ARP_poisioning	3.813601e+06
DDOS_Slowloris	2.760618e+06

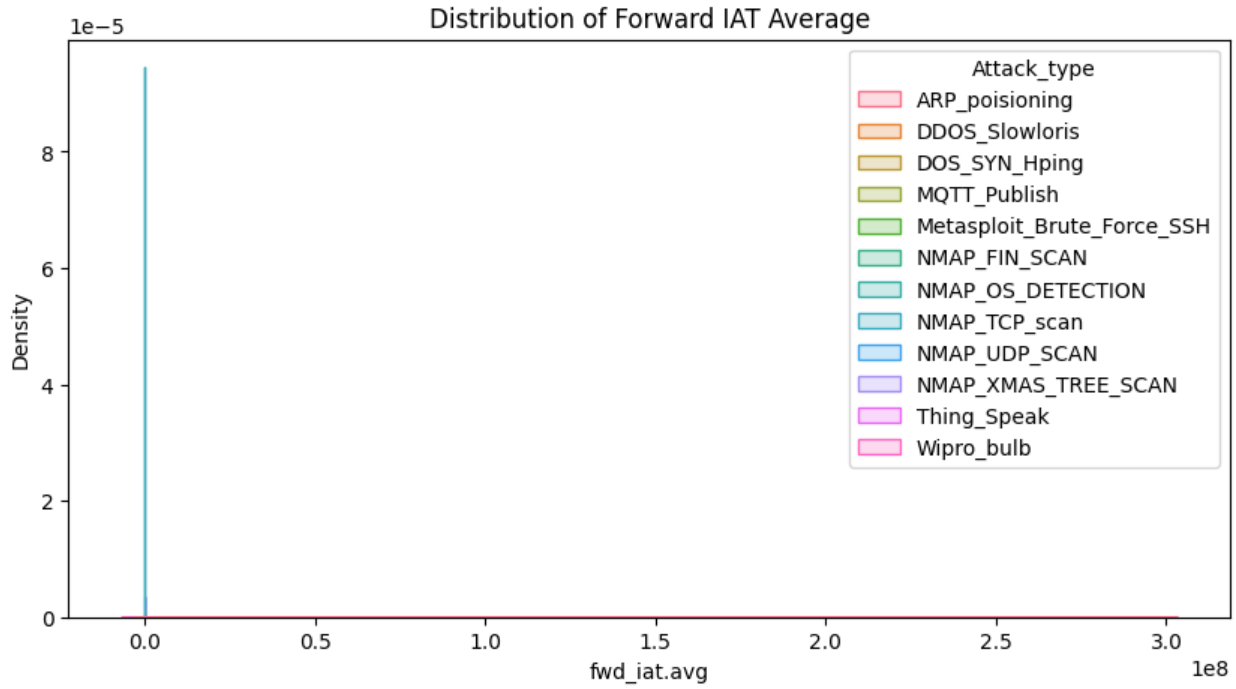
```
DOS_SYN_Hping                0.000000e+00
MQTT_Publish                 1.378185e+06
Metasploit_Brute_Force_SSH   4.747146e+06
NMAP_FIN_SCAN                2.776590e+04
NMAP_OS_DETECTION            0.000000e+00
NMAP_TCP_scan                0.000000e+00
NMAP_UDP_SCAN                4.287451e+03
NMAP_XMAS_TREE_SCAN          6.956765e+03
Thing_Speak                  5.627067e+05
Wipro_bulb                   6.598996e+06
```



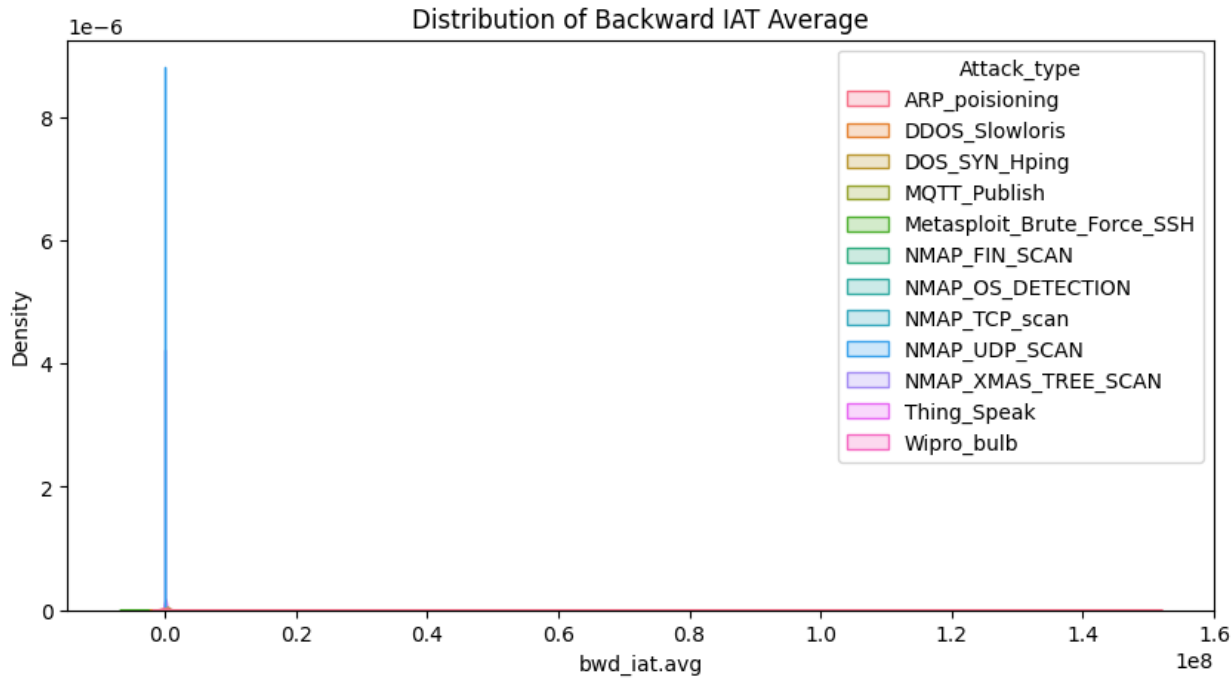

```
/tmp/ipykernel_12991/3025982171.py:27: FutureWarning: The default of
observed=False is deprecated and will be changed to True in a future
version of pandas. Pass observed=False to retain current behavior or
observed=True to adopt the future default and silence this warning.
  mean_values = df.groupby('Attack_type')[time_features].mean()
```




```
/tmp/ipykernel_12991/3025982171.py:38: UserWarning: Dataset has 0
variance; skipping density estimate. Pass `warn_singular=False` to
disable this warning.
  sns.kdeplot(data=df, x='fwd_iat.avg', hue='Attack_type', fill=True)
```



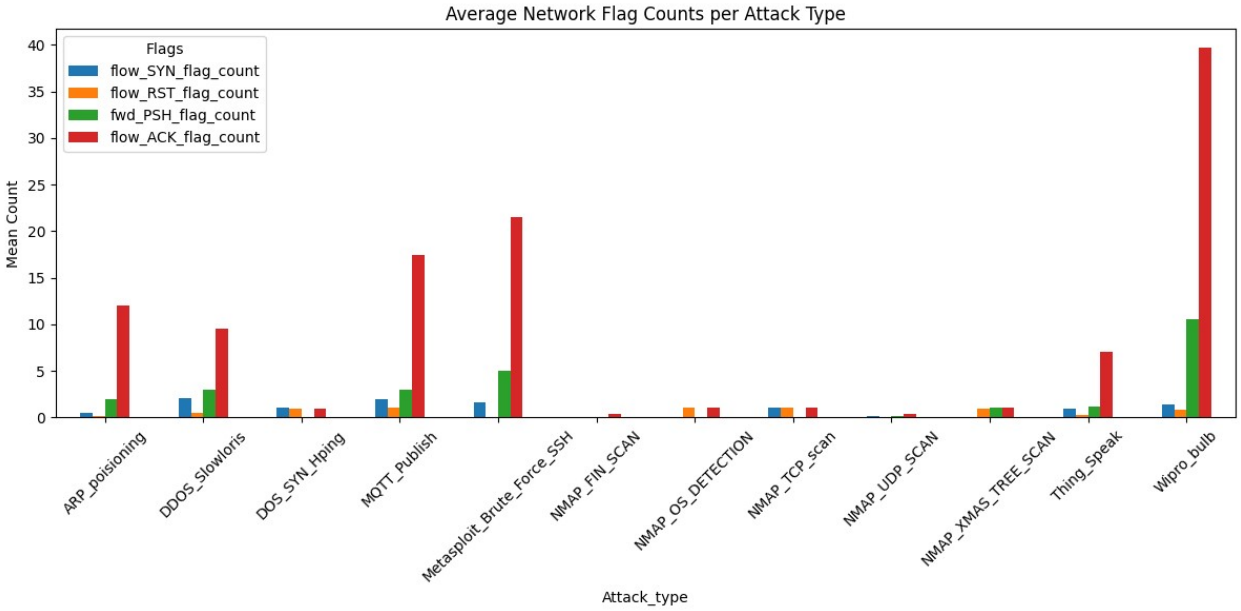

```
/tmp/ipykernel_12991/3025982171.py:43: UserWarning: Dataset has 0
variance; skipping density estimate. Pass `warn_singular=False` to
disable this warning.
  sns.kdeplot(data=df, x='bwd_iat.avg', hue='Attack_type', fill=True)
```

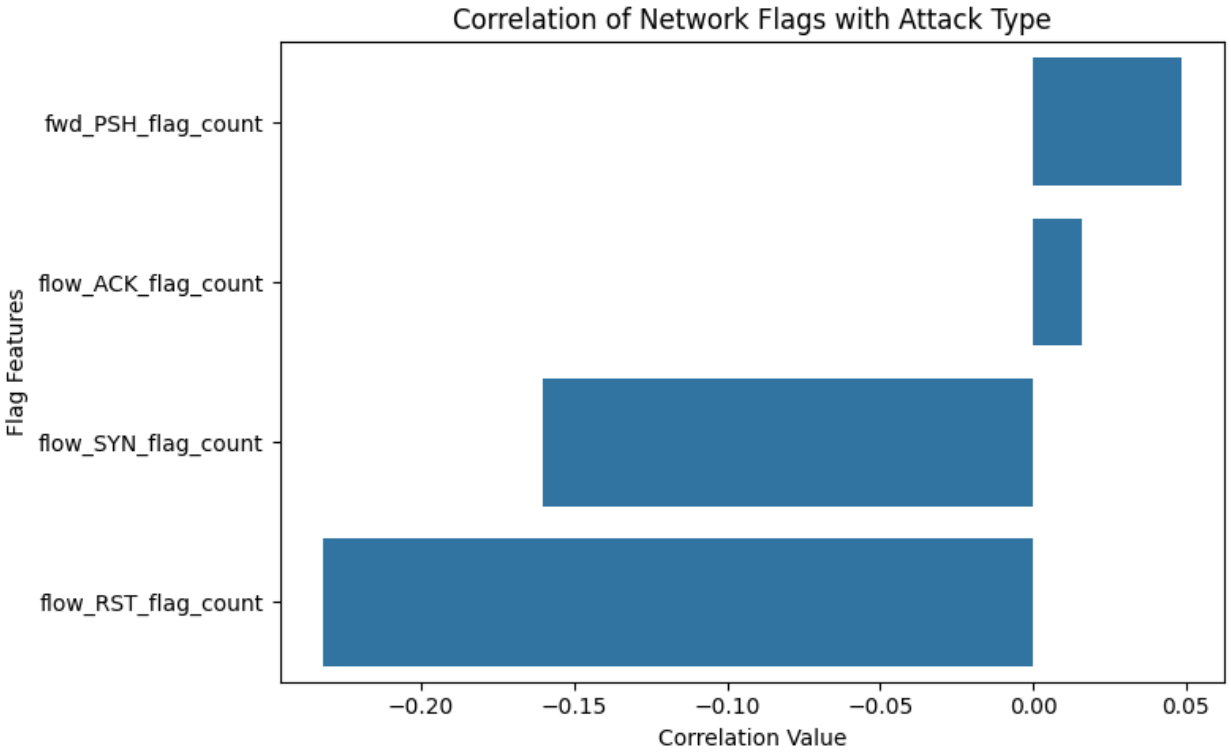


Which network flag counts (e.g., flow_SYN_flag_count, flow_RST_flag_count, fwd_PSH_flag_count) are most indicative of specific intrusion patterns?

```
flag_features = [
    'flow_SYN_flag_count',
    'flow_RST_flag_count',
    'fwd_PSH_flag_count',
    'flow_ACK_flag_count'
]

flag_stats = df.groupby('Attack_type')[flag_features].mean()

flag_stats.plot(kind='bar', figsize=(12,6))
plt.title('Average Network Flag Counts per Attack Type')
plt.ylabel('Mean Count')
plt.xticks(rotation=45)
plt.legend(title="Flags")
plt.tight_layout()
plt.show()

le = LabelEncoder()
df['Attack_type_encoded'] = le.fit_transform(df['Attack_type'])
flag_corr = df[flag_features + ['Attack_type_encoded']].corr()
corr_values =
flag_corr['Attack_type_encoded'].drop('Attack_type_encoded').sort_valu
es(ascending=False)

plt.figure(figsize=(8,5))
```

```
sns.barplot(
    x=corr_values.values,
    y=corr_values.index
)

plt.title("Correlation of Network Flags with Attack Type")
plt.xlabel("Correlation Value")
plt.ylabel("Flag Features")

plt.tight_layout()
plt.show()
```




Correlation of Network Flags with Attack Type
fwd_PSH_flag_count
flow_ACK_flag_count
flow_SYN_flag_count
flow_RST_flag_count
Flag Features
−0.20
−0.15
−0.10
−0.05
0.00
0.05
Correlation Value